4

Unit 4: Sequential Logic Circuits

Digital Logic Design

CHANDRAS EDU

Never stop learning

Outline

4.1 Latches
4.2 Flip-Flops & Timing
4.3 Asynchronous (Ripple) Counters
4.4 Synchronous Counters
4.5 Shift Registers & Applications

4.1 Latches

IN SIMPLE TERMS

A latch is two gates feeding each other, so it remembers one bit; with an enable input, it copies D only while EN = 1.

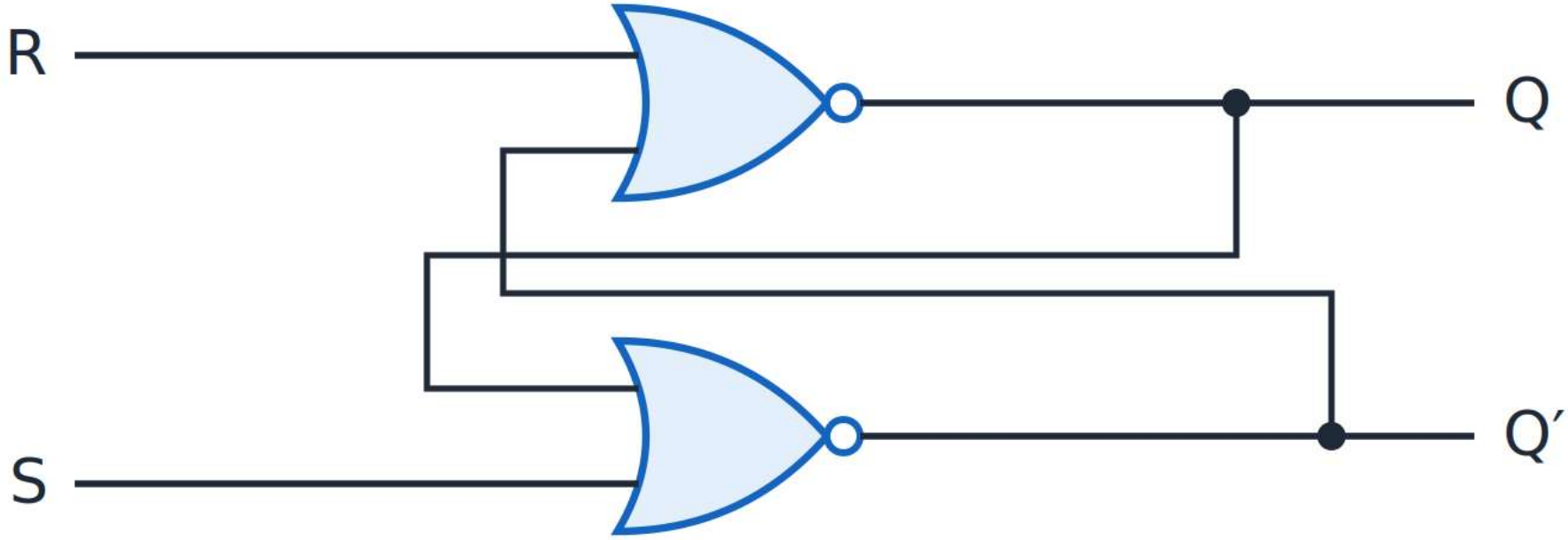
Combinational versus sequential logic

- Units 1–3 were **combinational**: output depends only on the present inputs, with no memory
- A **sequential circuit** adds memory: its output depends on the present inputs **and** the current **state**
- The state comes from feeding an output signal back into the input logic
- **Next State (Q_{next}) = f(Inputs, Current State Q)**
- A **latch** is the simplest sequential element: a 1-bit memory cell built from ordinary gates wired with feedback

The SR latch (Set-Reset)

- Cross-couple two NOR gates: each gate's output feeds back as one of the other gate's inputs
- The circuit has **two stable states**, which is exactly what is needed to remember one bit
- Top gate: **Q = NOR(R, Q')**. Bottom gate: **Q' = NOR(S, Q)**
- The two feedback paths cross, which is why S ends up wired to the gate that produces Q', not Q

SR latch



SR latch with two cross-coupled NOR gates.

SR latch functionality

S (Set)	R (Reset)	Q_{next}	State	Behavior
0	0	Q	**Hold**	No change: holds current state
1	0	1	**Set**	Forces Q to 1
0	1	0	**Reset**	Forces Q to 0
1	1	Invalid	**Forbidden**	Both Q and Q’ driven to 0, breaking the complement relationship. Must be avoided

Worked example: tracing S = 1, R = 0

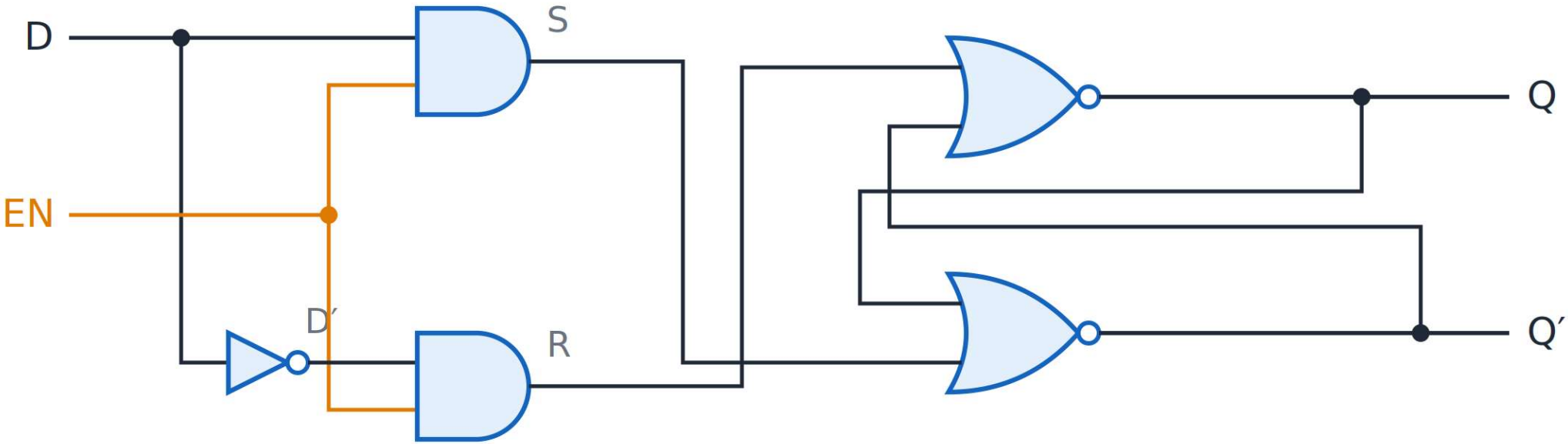

```
Bottom gate: S = 1 → Q′ = NOR(1, anything) = 0
Feed Q′ = 0 back into the top gate with R = 0
Top gate: Q = NOR(0, 0) = 1
So S = 1 forces Q = 1
```

- S = 1 first forces the other gate’s output low, then that value feeds forward

The gated D latch

- The SR latch has two problems: the forbidden S = R = 1 state, and no control over **when** it updates
- The **gated D latch** fixes both
- Drive the inputs from one data line D: **S = D** and **R = D'**; S and R are always complements, so S = R = 1 can never happen
- Add an **Enable (EN)** input: two AND gates give S = D·EN and R = D'·EN
- When EN = 0, both S and R are 0 and the SR latch holds its last state

Gated D latch



Gated D latch: S = D·EN and R = D′·EN drive the NOR latch, so S = R = 1 can never occur.

Transparent versus latched mode

- **EN = 1 (transparent):** Q follows D continuously; any change on D shows up on Q immediately
- **EN = 0 (latched):** Q holds its last value and ignores further changes on D
- While EN is high the output tracks D; dropping EN freezes whatever was there
- This circuit behaves as pure 1-bit data storage

4.2 Flip-Flops & Timing

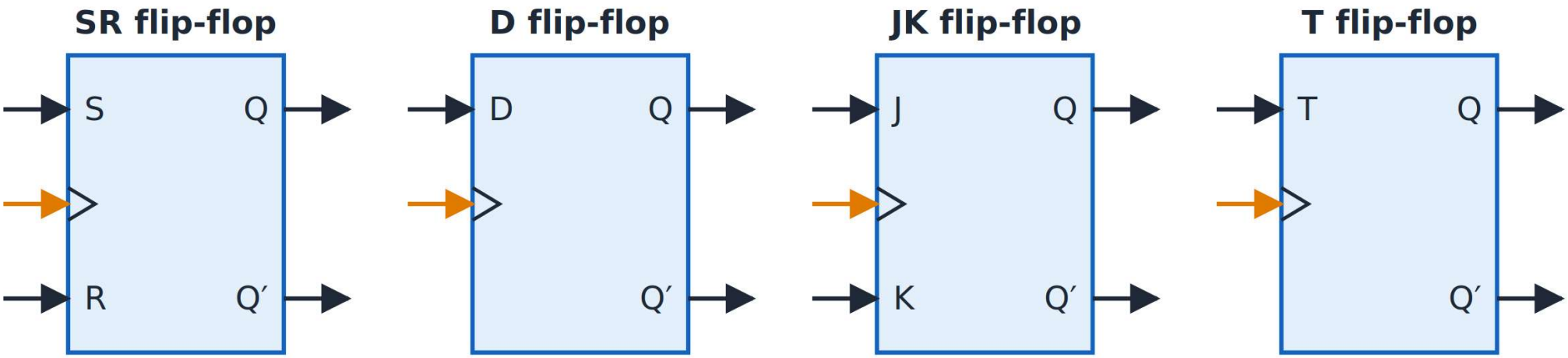
IN SIMPLE TERMS

A flip-flop is a latch that changes only at the clock edge, so every flip-flop in a system updates at the same instant.

Latch versus flip-flop

- The gated D latch is **level-sensitive**: for the entire time EN is high, Q tracks D
- A **flip-flop** is **edge-triggered**: it samples its inputs and updates Q only when the clock transitions (↑ rising or ↓ falling edge)
- It ignores its inputs the rest of the time
- That single change makes counters, registers and state machines reliable
- Every stage changes at the same defined instant, instead of drifting with how long a level stays high

Flip-flop graphic symbols



The triangle marks the clock input; the flip-flop changes only at the active clock edge

Graphic symbols of the SR, D, JK and T flip-flops; the triangle marks the clock input.

Characteristic equations and excitation tables

- A **characteristic equation** gives the next state Q_{n+1} as a function of the inputs and the present state Q_n
- An **excitation table** flips this around: for a desired $Q_n \rightarrow Q_{n+1}$ transition, which inputs are required
- Excitation tables link “what sequence of states do I want” to “how do I wire the inputs”
- They are the basis of synchronous counter design in the next topic

Characteristic and excitation tables (1 of 2)

Type	Characteristic equation	$Q_n \rightarrow Q_{n+1}$	Required input(s)
D	$Q_{n+1} = D$	$0 \rightarrow 0$	D = 0
		$0 \rightarrow 1$	D = 1
		$1 \rightarrow 0$	D = 0
		$1 \rightarrow 1$	D = 1
T	$Q_{n+1} = T \oplus Q_n$	$0 \rightarrow 0$	T = 0
		$0 \rightarrow 1$	T = 1
		$1 \rightarrow 0$	T = 1
		$1 \rightarrow 1$	T = 0

Characteristic and excitation tables (2 of 2)

Type	Characteristic equation	$Q_n \rightarrow Q_{n+1}$	Required input(s)
JK	$Q_{n+1} = JQ_n' + K'Q_n$	0 → 0	J = 0, K = X
		0 → 1	J = 1, K = X
		1 → 0	J = X, K = 1
		1 → 1	J = X, K = 0
SR	$Q_{n+1} = S + R'Q_n$ (SR = 0)	0 → 0	S = 0, R = X
		0 → 1	S = 1, R = 0
		1 → 0	S = 0, R = 1
		1 → 1	S = X, R = 0

Why don’t-cares matter

- The **X** (don’t-care) entries are the point of using JK or T over D in counter design
- Whenever an input can be 0 or 1 without affecting the transition, a later K-map step can exploit that freedom
- **D** has no don’t-cares: every transition pins down D exactly
- **SR** has two (0 → 0 and 1 → 1), fewer than JK
- S and R must never both be 1: the forbidden state inherited from the latch

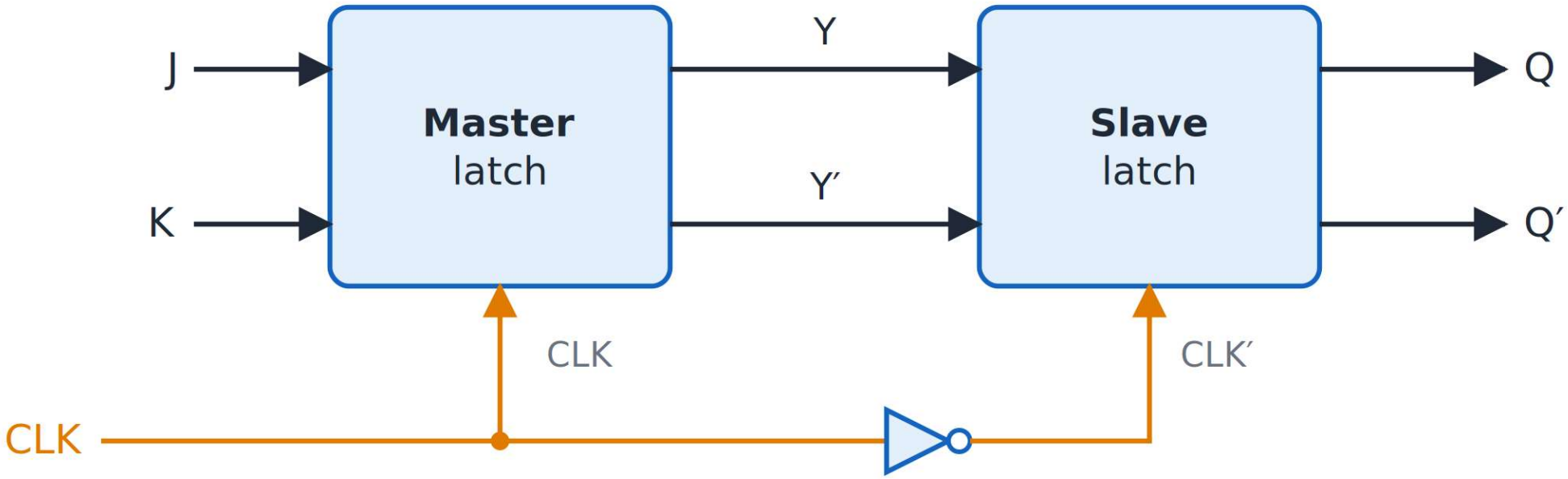
The race-around problem

- A level-triggered JK latch with **J = K = 1** should toggle
- If the clock pulse stays high longer than the gate propagation delay t_{pd}, the output toggles again, and again
- The final state depends on how many times it toggled: unpredictable
- This is the **race-around condition**

The master-slave solution

- Chain **two** latches, a master and a slave, controlled by opposite phases of the clock
- The two halves are never transparent at the same time
- **CLK = 1:** the master is transparent and absorbs J, K into its state Y; the slave is latched and isolates Q
- **CLK = 0:** the master locks Y; the slave becomes transparent and copies Q = Y
- Q changes only on the falling edge; the master is already locked, so there is **no race**

Master-slave flip-flop

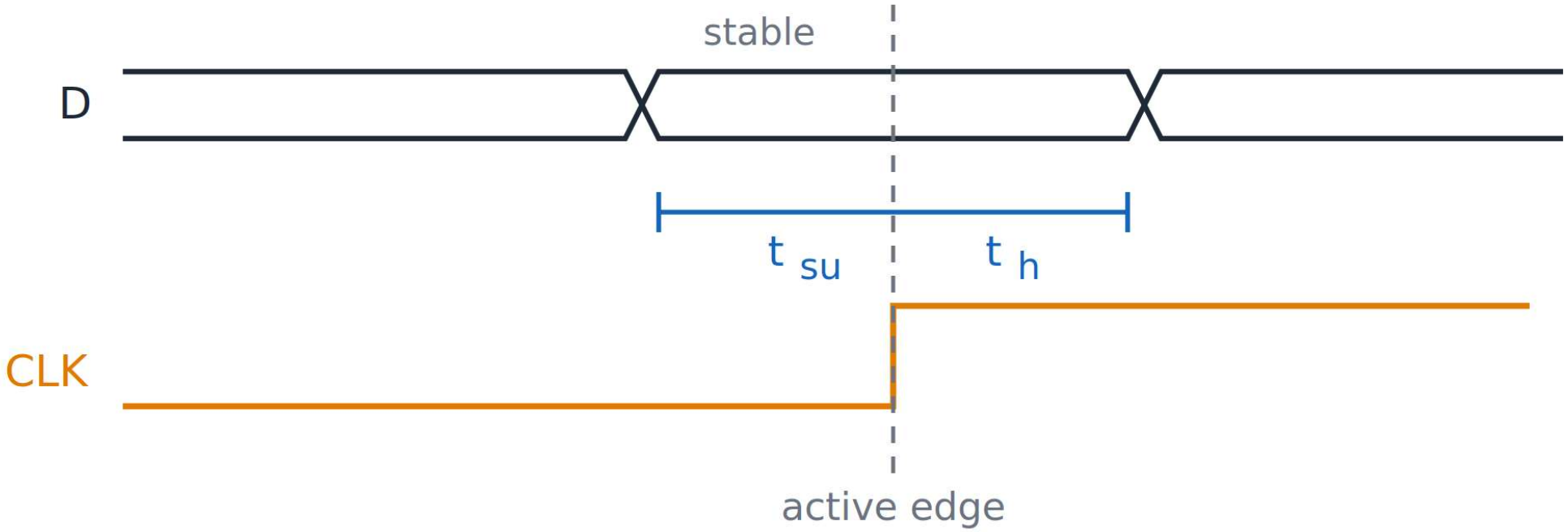


Master-slave flip-flop: the master follows J, K while CLK = 1; the slave copies it while CLK = 0.

Setup time, hold time and clock skew

- **Setup time (t_{su}):** the minimum time D must be stable **before** the active clock edge
- **Hold time (t_h):** the minimum time D must stay stable **after** the active clock edge
- **Clock skew:** the difference in clock arrival time at different flip-flops, caused by unequal interconnect delay
- Skew eats into the setup and hold margin
- Violating setup or hold time risks **metastability**: the output hovers between 0 and 1 before resolving unpredictably

Setup and hold time



Setup time t_{su} and hold time t_h around the active clock edge.

4.3 Asynchronous (Ripple) Counters

IN SIMPLE TERMS

In a ripple counter each flip-flop toggles the next one, so a count change ripples down the chain instead of happening all at once.

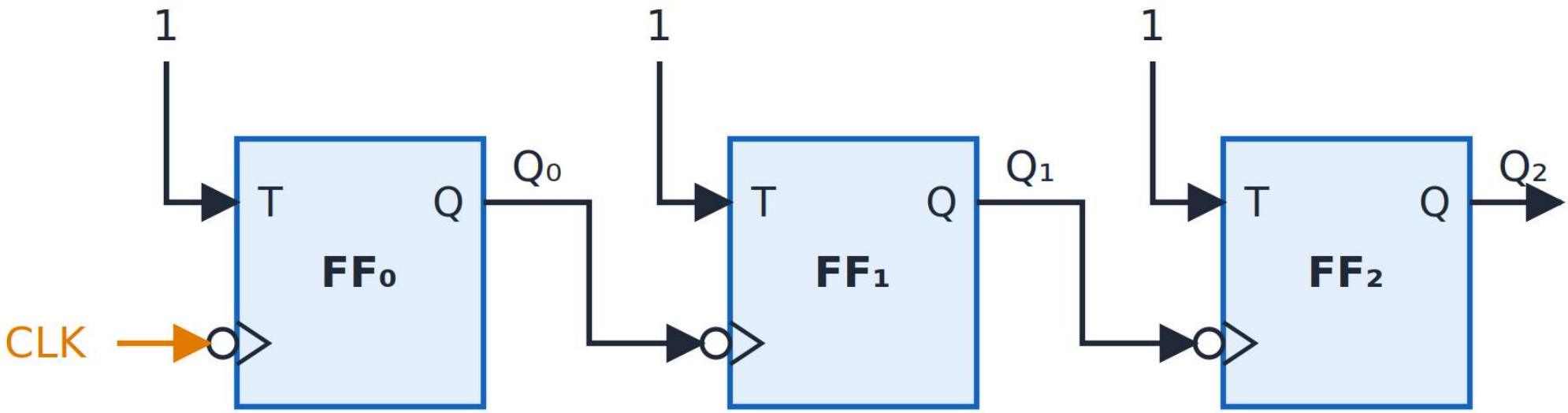
What makes a counter asynchronous?

- A counter is a sequential circuit that cycles through a fixed sequence of binary states on every clock pulse
- In an **asynchronous (ripple)** counter only the **first** flip-flop receives the external clock
- Every later stage is clocked by the output of the stage before it
- The new count “ripples” through the chain one stage at a time

3-bit asynchronous up-counter

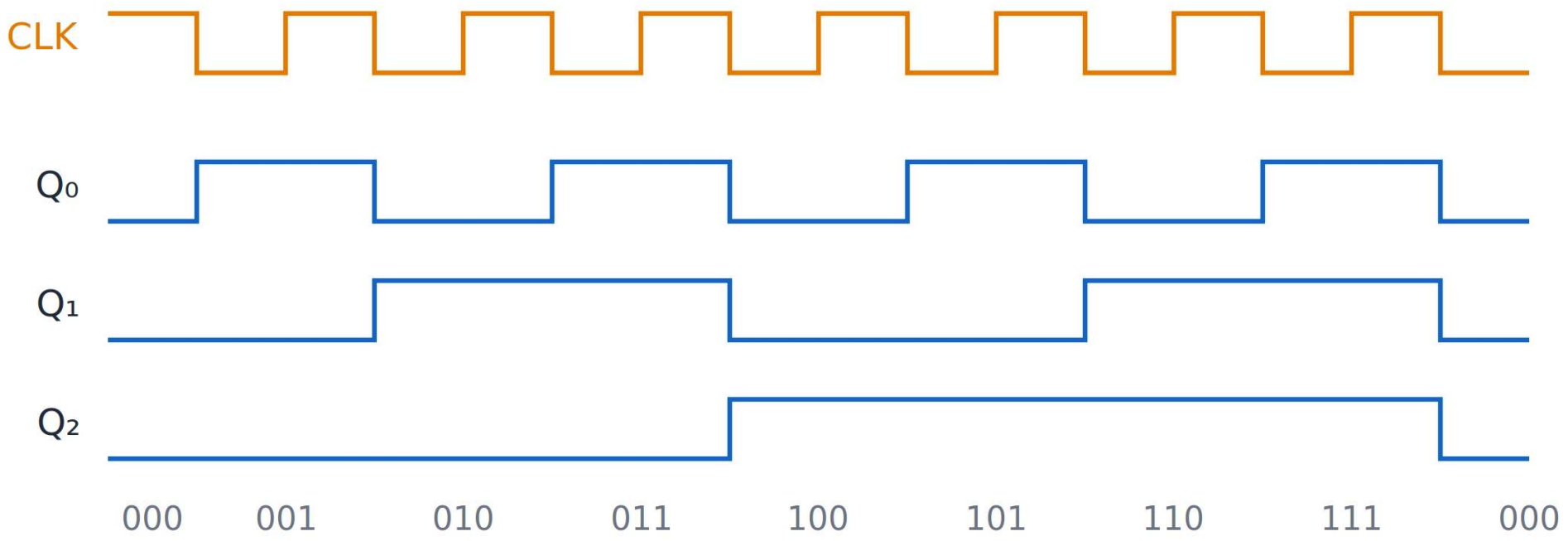
- Cascade three negative-edge triggered **T flip-flops**, each wired **T = 1** (toggle every clock)
- **FF_0** is the LSB and toggles on every external clock pulse
- **FF_1** toggles only when Q_0 falls; **FF_2** only when Q_1 falls
- Each stage runs at half the frequency of the one before: exactly binary counting
- Sequence: 000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000 …

3-bit ripple up-counter



Three-bit ripple up-counter: negative-edge T flip-flops, each clocked by the previous Q.

Ripple counter timing



Timing of the ripple counter: each output toggles on the falling edge of the one before, so $Q_2Q_1Q_0$ counts 000 to 111.

Up and down counter rules

Up or down depends on two choices: the clock edge each flip-flop triggers on, and whether the next stage is driven by Q or Q′. Negative-edge + Q, or positive-edge + Q′, count up; the other two count down.

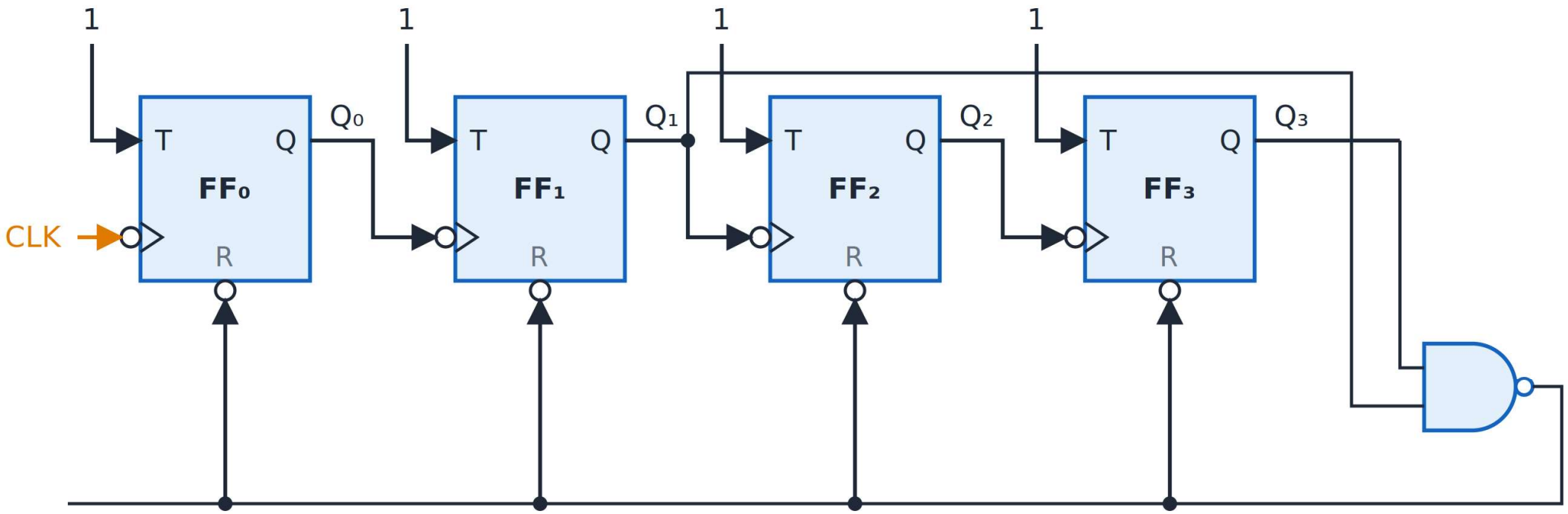
Clock triggering	Next stage driven by	Counter mode
Negative edge (↓)	Q	**UP**
Negative edge (↓)	Q′	**DOWN**
Positive edge (↑)	Q	**DOWN**
Positive edge (↑)	Q′	**UP**

Designing a modulo-10 (BCD) counter

Required FFs: $2^n \geq 10 \implies n = 4$ flip-flops ($Q_3Q_2Q_1Q_0$)
Reset state: $Q_3Q_2Q_1Q_0 = 1010$ ($Q_3 = 1$ and $Q_1 = 1$)
Clear logic: $CLR' = (Q_3 \cdot Q_1)'$

- A plain 4-bit ripple counter is mod-16 (0000 through 1111)
- It must reset to 0000 the instant it would reach 10 (1010_2)
- At 1010, Q_3 and Q_1 are both 1, so the NAND output CLR’ drops to 0 and asynchronously clears every flip-flop
- The count never settles at 1010: it flashes through it as a brief reset trigger

Mod-10 ripple counter



CLR′: clears all four stages when the count reaches 1010

Mod-10 ripple counter: the NAND gate detects 1010 ($Q_3 = Q_1 = 1$) and clears all four stages.

4.4 Synchronous Counters

IN SIMPLE TERMS

In a synchronous counter every flip-flop hears the same clock tick; gates decide which ones toggle, so all bits change together.

What is a synchronous counter?

- Every flip-flop connects to the **same common clock**
- Every stage samples and updates at the exact same instant, instead of rippling through
- This removes the accumulated propagation delay of a ripple counter
- The cost: extra combinational logic to work out each stage’s inputs, so all stages land on the correct next state together

The design procedure

- **State table:** map every present state to its required next state
- **Excitation table:** apply each flip-flop type's excitation rules (previous topic) to find the inputs each transition requires
- **K-map minimisation:** treat each flip-flop input as its own Boolean function of the present-state variables and simplify
- **Circuit diagram:** wire the simplified gate logic to each flip-flop's inputs and tie every clock input to the same common line

3-bit up-counter with JK flip-flops

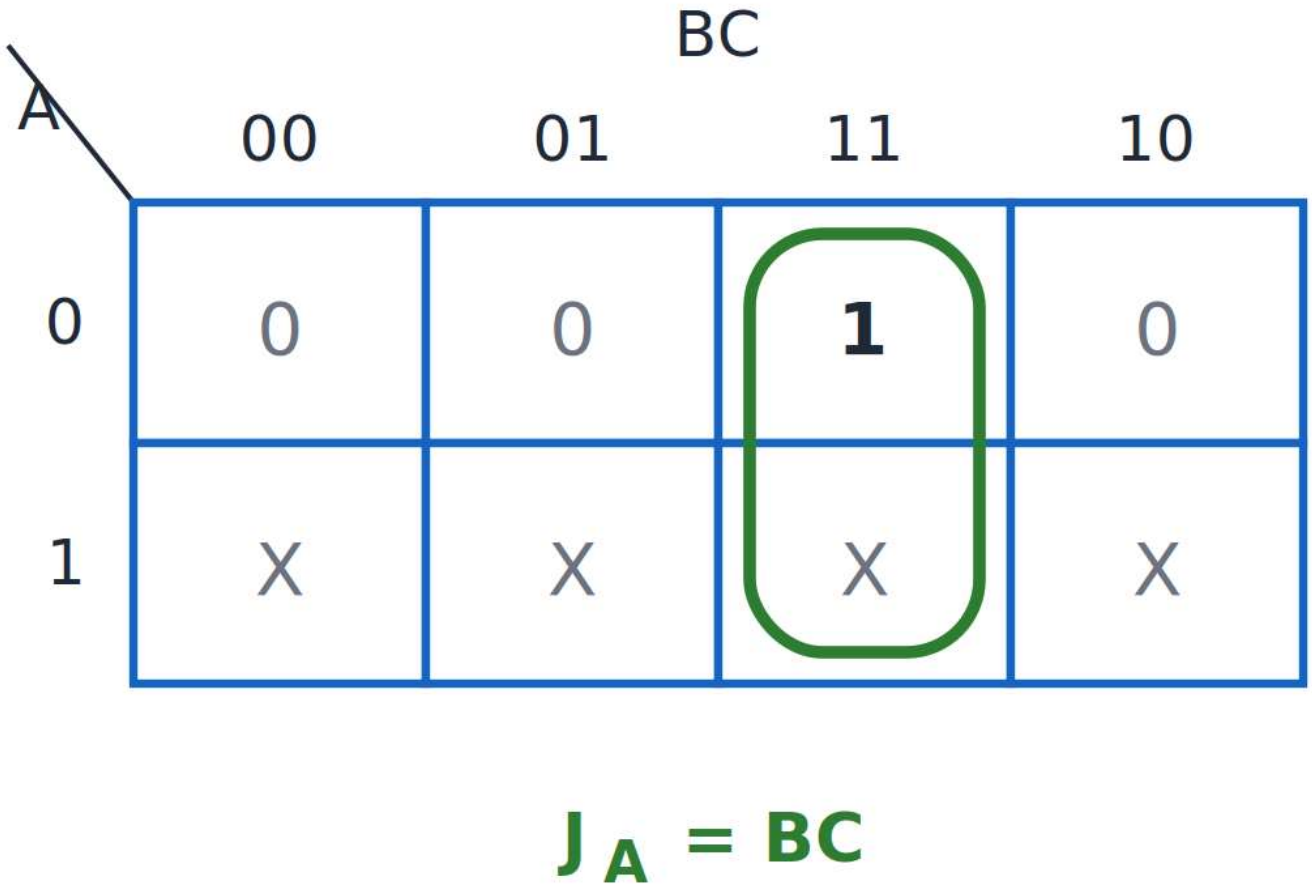
Three JK flip-flops, state $A_nB_nC_n$ (A = MSB), counting 000 through 111. Steps 1 and 2 combined: the required next state and the JK excitation values.

$A_nB_nC_n$	$A_{n+1}B_{n+1}C_{n+1}$	J_A,K_A	J_B,K_B	J_C,K_C
000	001	0, X	0, X	1, X
001	010	0, X	1, X	X, 1
010	011	0, X	X, 0	1, X
011	100	1, X	X, 1	X, 1
100	101	X, 0	0, X	1, X
101	110	X, 0	1, X	X, 1
110	111	X, 0	X, 0	1, X
111	000	X, 1	X, 1	X, 1

Step 3: K-maps for J and K of flip-flop A

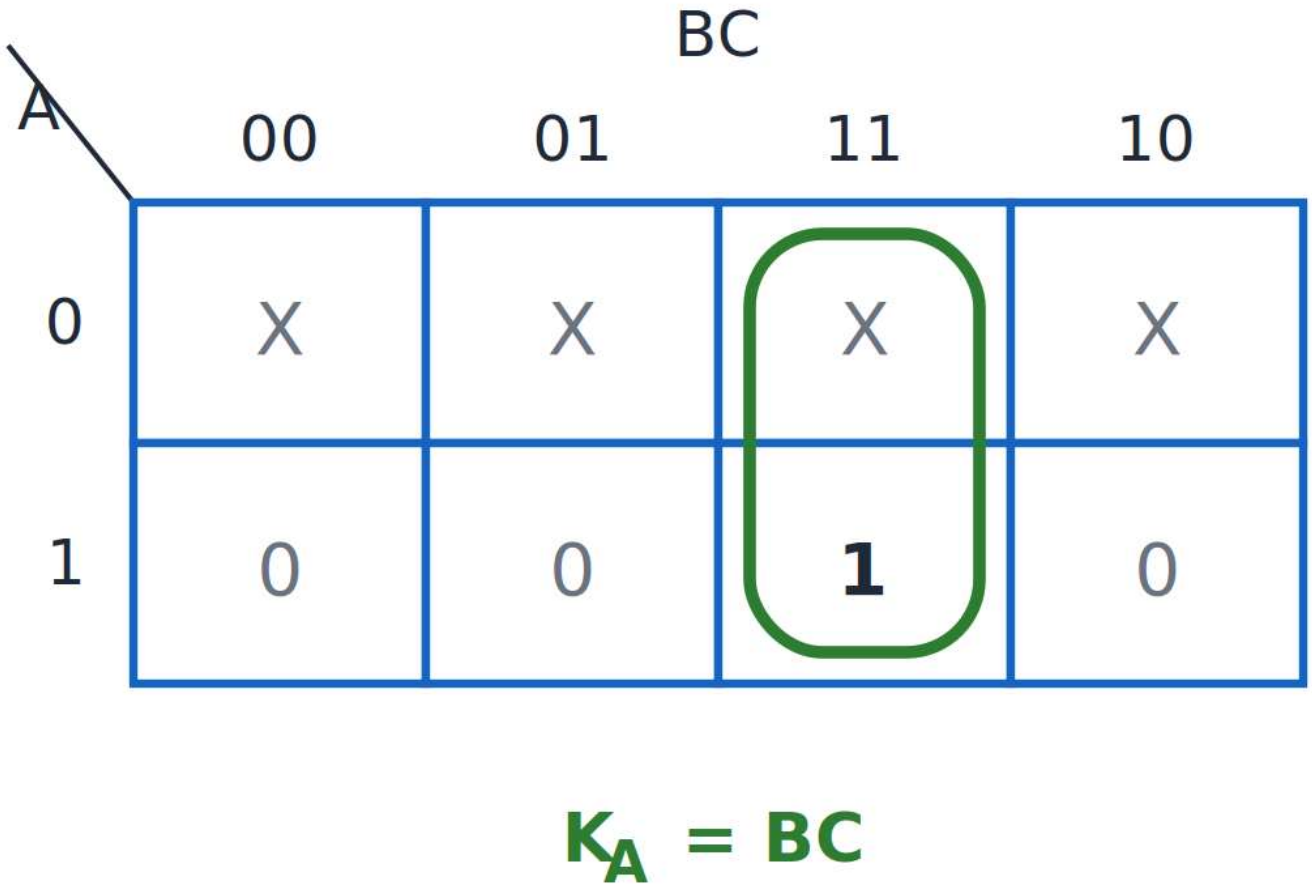
- Read the J_A and K_A columns onto 3-variable K-maps (rows = A, columns = BC in Gray-code order)
- Every A = 1 entry is a don’t-care in both maps
- That leaves a single 1 in each: ABC = 011 for J_A and ABC = 111 for K_A
- Each 1 is free to pair up with the don’t-care directly below it
- Both maps group the same pair (BC = 11, both A rows): the constant variables are B = 1 and C = 1, so $\mathbf{J_A = K_A = BC}$

K-map for J of flip-flop A



K-map for J_A: $J_A = BC$.

K-map for K of flip-flop A



K-map for K_A: $K_A = BC$.

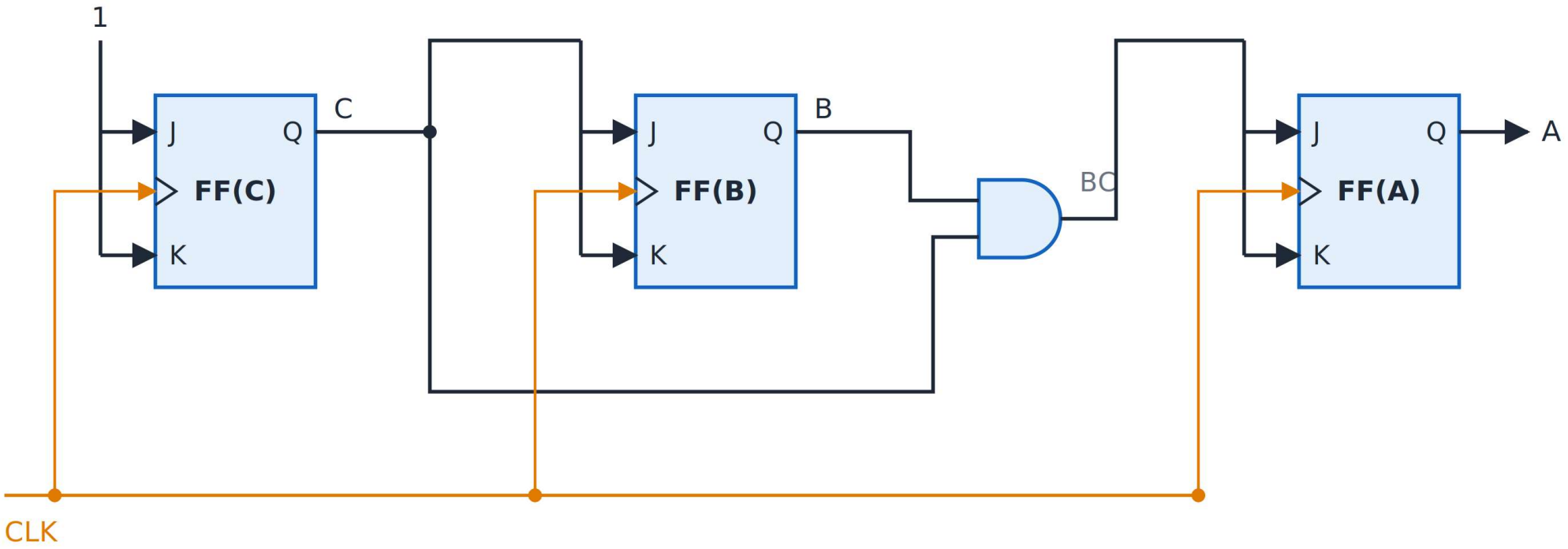
Stages B and C, by inspection

- The other two stages can be read directly from the excitation table without separate maps
- **$J_C = 1, K_C = 1$:** C toggles every single clock, just like the LSB of the ripple counter
- **$J_B = C, K_B = C$:** B's J/K columns are 0 when C = 0 and 1 or X when C = 1, so B toggles precisely when C is 1

Step 4: the circuit

- Every clock triangle taps the same horizontal **CLK line**: the structural difference from the ripple counter's staggered chain
- **FF(C)** always toggles
- **FF(B):** J and K both come straight from C
- **FF(A):** J and K both come from one AND gate combining B and C
- Feeding the same signal to both J and K means "toggle when this condition is true"

3-bit synchronous up-counter



Three-bit synchronous up-counter: J and K of each flip-flop are tied together; all clocks share CLK.

4.5 Shift Registers & Applications

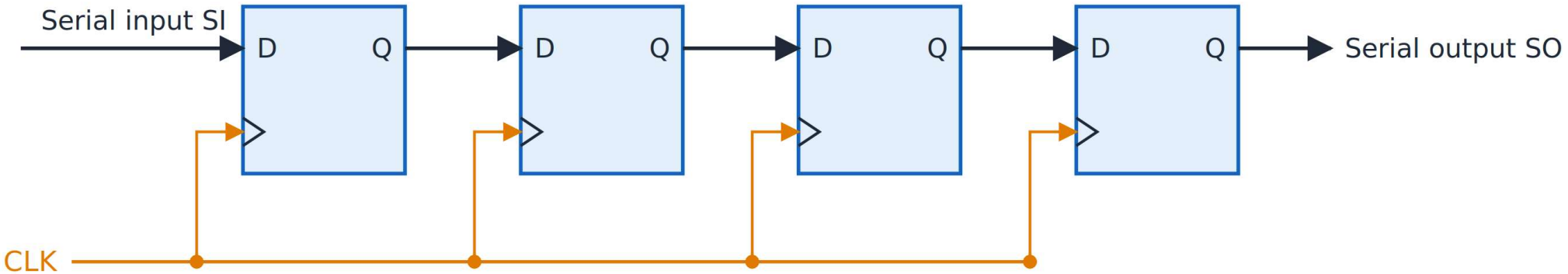
IN SIMPLE TERMS

A shift register passes its bits one place along on every clock, like a bucket brigade for bits; feed the end back to the start and it becomes a counter.

What is a shift register?

- A **shift register** is a cascade of flip-flops sharing one common clock
- Each stage's D input is simply the **previous stage's Q**
- On every clock pulse every stored bit moves one position down the chain
- Serial or parallel on each side is purely about how data gets in and out; the internal shifting structure never changes

Four-bit shift register



Four-bit shift register: each Q feeds the next D on every clock edge. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 6.3.

Four data transfer modes

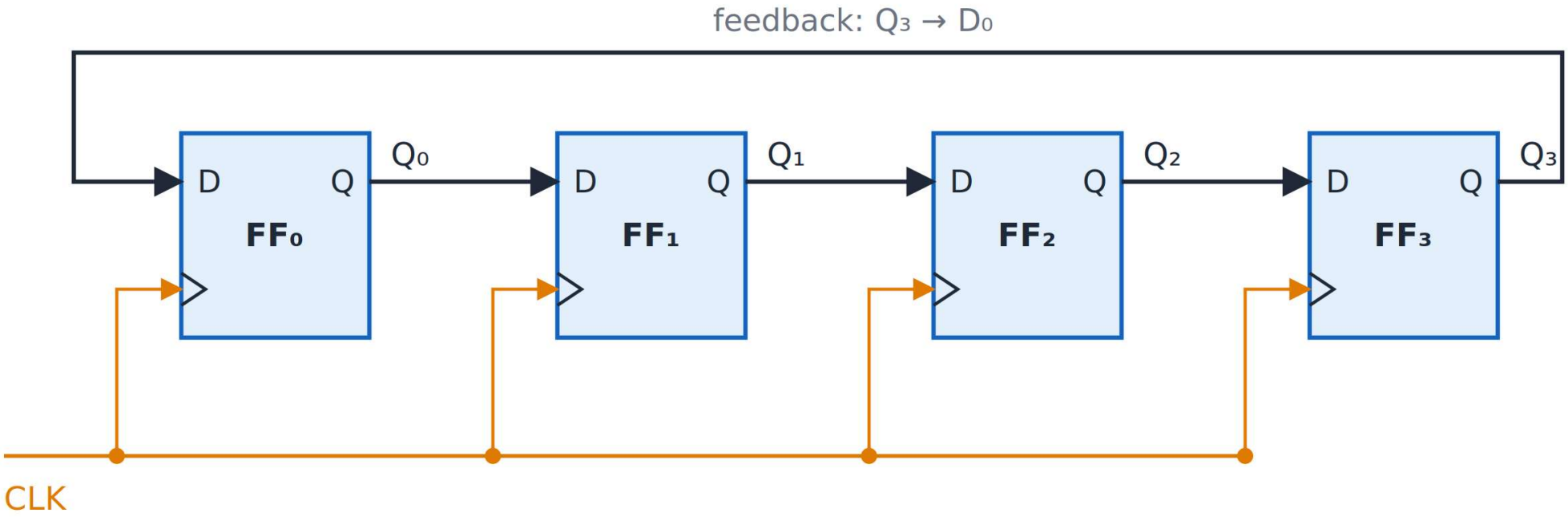
SIPO taps every stage's Q as a separate output pin; PISO adds a way to force each stage to an external bit before shifting. Both are the same shift-register core with extra access points.

Type	Name	Data in	Data out	Typical use
SISO	Serial-In / Serial-Out	1 bit/clock	1 bit/clock	Time delay, signal buffering
SIPO	Serial-In / Parallel-Out	1 bit/clock	All bits at once	Serial-to-parallel conversion (UART receive)
PISO	Parallel-In / Serial-Out	All bits at once	1 bit/clock	Parallel-to-serial conversion (UART transmit)
PIPO	Parallel-In / Parallel-Out	All bits at once	All bits at once	Fast temporary register storage

Ring counter

- Feed the last stage's output straight back into the first stage's input, with no external serial data
- The register becomes a self-contained counter: a **ring counter**
- **Number of states:** N states for N flip-flops; the single 1 (or single 0) just circulates
- **4-bit sequence, pre-loaded to 1000:** 1000 → 0100 → 0010 → 0001 → 1000 …
- **Must be pre-loaded:** reset alone gives 0000, a stuck state ($Q_3 = 0$ feeds $D_0 = 0$ forever)

Four-bit ring counter

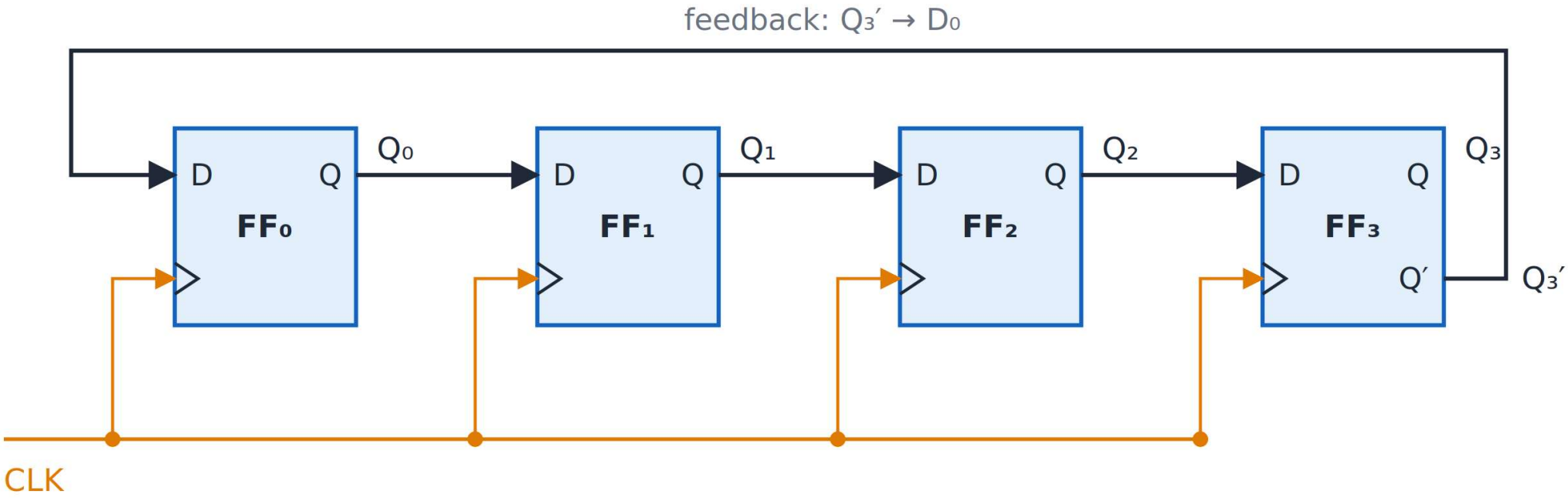


Four-bit ring counter: Q_3 feeds back to D_0.

Johnson counter (twisted-ring counter)

- Feed back the **inverted** last output instead of the plain one
- The circulating bit becomes a spreading and retreating band of 1s and 0s
- **Number of states:** 2N states for N flip-flops: twice the ring counter
- **4-bit sequence from reset (0000):** 0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000 …
- **Self-starting** from 0000: $Q_3' = 1$ feeds $D_0 = 1$ on the very next clock

Four-bit Johnson counter



Four-bit Johnson counter: Q_3' feeds back to D_0.

Ring versus Johnson counter

Feature	Ring counter	Johnson counter
Number of states	N	2N
Feedback connection	$D_0 = Q_{N-1}$	$D_0 = Q'_{N-1}$
Initialization	Requires pre-loading (e.g., 1000)	Self-starting from reset (0000)
Decoding next state	Single bit is already the decoded output	Needs a 2-input gate per state to decode

Unit 4 summary

- **4.1 Latches:** cross-coupled NOR gates hold a bit; the gated D latch is level-sensitive
- **4.2 Flip-flops and timing:** edge-triggered; characteristic and excitation tables; master-slave; setup and hold
- **4.3 Asynchronous counters:** each stage clocks the next; up or down by edge and Q or Q’; mod-10 by NAND reset
- **4.4 Synchronous counters:** one shared clock; state table, excitation table, K-map, circuit
- **4.5 Shift registers:** SISO, SIPO, PISO, PIPO; ring counter (N states) and Johnson counter (2N states)

Review questions (1 of 2)

- **Q1.** Explain the difference between a level-sensitive latch and an edge-triggered flip-flop
- **Q2.** Why is S = R = 1 forbidden in an SR latch, and how does the gated D latch avoid it?
- **Q3.** What is the race-around condition and how does the master-slave structure fix it?
- **Q4.** Write the excitation table entries for a JK flip-flop and explain the don’t-cares

Review questions (2 of 2)

- **Q5.** Which combinations of clock edge and Q or Q’ give an up-counter in a ripple counter?
- **Q6.** Design a modulo-10 ripple counter: how many flip-flops and what reset logic?
- **Q7.** Derive J_A and K_A for a 3-bit synchronous up-counter using K-maps
- **Q8.** Compare ring and Johnson counters for N flip-flops

References

- **Textbook:** M. Morris Mano and Michael D. Ciletti, Digital Design
- **Adapted figure:**
- Four-bit shift register (Fig. 6.3)